

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВАСТОПОЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
ВТШ «СПИ»

ОТЧЕТ
о выполнении лабораторной работы № 1
Тема: «МЕТОДЫ ОДНОМЕРНОЙ
МИНИМИЗАЦИИ УНИМОДАЛЬНОЙ
ФУНКЦИИ»
по дисциплине
«ПРИКЛАДНАЯ МАТЕМАТИКА В ИИ»
Вариант № 4

Выполнил:

Студент группы ИИ/б-25-6-о

Заварзин А. В.

Проверила:

к.т.н., доцент, доцент кафедры ПМии,
доцент кафедры ИТиС

Ченгарь О. В.

Севастополь, 2026

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. В MS Excel построить таблицу значений заданной функции и её график на произвольном отрезке.
2. На основании построенного графика определить отрезок $[a; b]$ с предполагаемым экстремумом функции (минимум функции $f(x)$).
3. Определить локальный минимум функции на основе методов одномерной оптимизации, выполнив 3-4 итерации (с помощью MS Excel):
 - а) методом деления отрезка пополам;
 - б) методом золотого сечения (для нечётных вариантов);
 - в) методом Фибоначчи (для чётных вариантов);
4. Решить задачу всеми методами, используя Python.
5. Сравнить результаты расчётов, сделанные в MS Excel и Python

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ ПО ВАРИАНТУ

Для варианта № 4 дана следующая функция:

$$f(x) = x^4 + 8x^3 - 6x^2 - 72x + 90$$

Для вычислений также возьмём значение погрешности:

$$\varepsilon = 0,05$$

ХОД РАБОТЫ

Задание 0

Из данной для варианта №4 функции найдём точки экстремума.

Для начала найдём производную функции:

$$f'(x) = 4x^3 + 24x^2 - 12x - 72$$

Далее приравниваем производную к нулю:

$$4x^3 + 24x^2 - 12x - 72 = 0$$

Находим корни:

$$\begin{cases} x_1 = -6 \\ x_2 = -\sqrt{3} \\ x_3 = \sqrt{3} \end{cases}$$

Подставляя все полученные значения в исходную функцию можно сказать, что точка минимума функции по оси располагается в -6. А значение функции в точке минимума равно -126.

Задание 1.

В начале необходимо построить таблицу значений функции в MS Excel, а затем, по полученной таблице, построить её график.

Результат построения таблицы значения и графика функции в среде MS Excel представлен на рисунке 2.1.

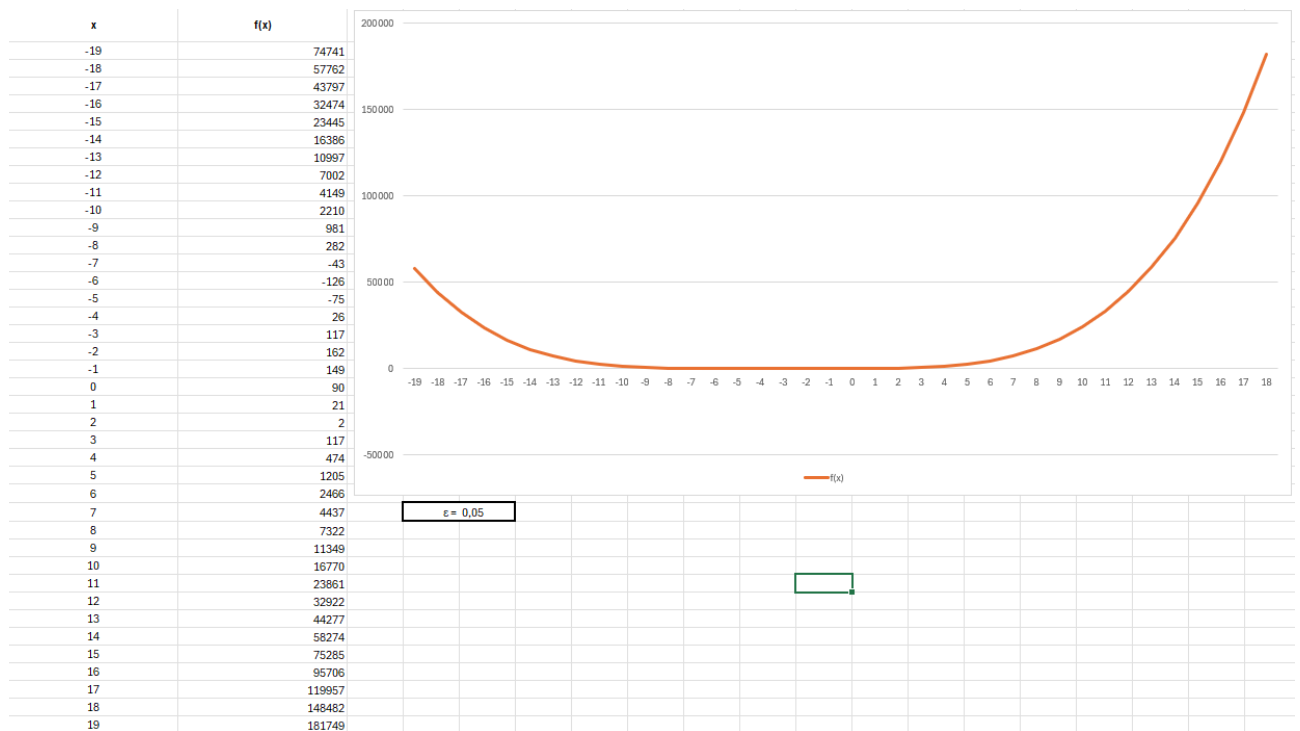


Рисунок 2.1 – Построенная таблица значений и график для функции

Задание 2

Исходя из полученного графика в задании 1 можно сказать, что минимум функции лежит в интервале $[-10, 6]$.

Задание 3

После определения интервала в котором лежит предполагаемый минимум необходимо пройти 3 итерации с помощью методов деления отрезка пополам и Фибоначчи. Сначала был произведён поиск минимума функции с помощью метода деления отрезка пополам. Результаты первых 3 итераций представлены на рисунке 2.2.

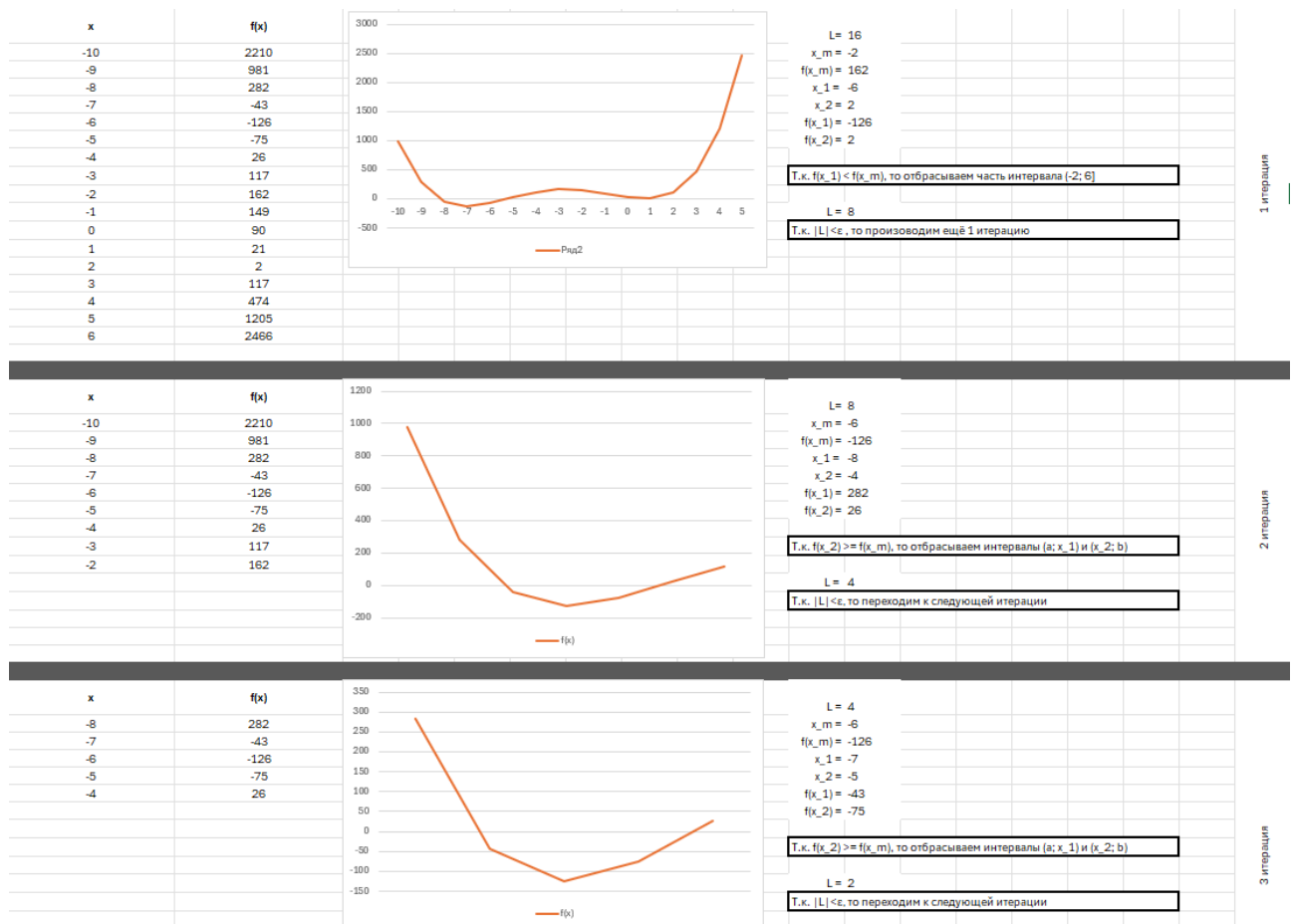


Рисунок 2.2 – Результаты первых 3 итераций метода деления отрезка пополам

Для расчётов были использованы инструменты MS Excel, а именно: обращение к элементам через ссылки, использование функций MS Excel для расчёта значения функции (функция СТЕПЕНЬ).

После произведения расчётов по методу деления отрезка пополам необходимо произвести 3 итерации по методу Фибоначчи.

Перед произведением вычислений по методу Фибоначчи необходимо построить таблицу чисел Фибоначчи. Часть таблицы чисел Фибоначчи, построенная средствами MS Excel представлена на рисунке 2.3.

1	F_0
1	F_1
2	F_2
3	F_3
5	F_4
8	F_5
13	F_6
21	F_7
34	F_8
55	F_9
89	F_10
144	F_11
233	F_12
377	F_13
610	F_14
987	F_15
1597	F_16
2584	F_17
4181	F_18
6765	F_19
10946	F_20
17711	F_21
28657	F_22
46368	F_23
75025	F_24
121393	F_25

Рисунок 2.3 – Часть таблицы чисел Фибоначчи

После построения таблицы чисел Фибоначчи можно произвести поиск минимума функции по методу Фибоначчи. Первые 4 итерации метода Фибоначчи представлены на рисунке 2.4.

x	f(x)	a = -10				
-10	2210	b = 6				
-9	981	L = 16				
-8	282	F_{n-1} = 320				
-7	-43	F_12 = 233				
-6	-126	F_13 = 377				
-5	-75	n = 14				
-4	26	x_1 = -3,88854				
-3	117	x_2 = -0,11146				
-2	162	f(x_1) = 37,50627				
-1	149	f(x_2) = 97,93959				
0	90	b = -0,11146				
1	21	x_2 = -3,88854				
2	2	n = 13				
3	117	x_1 = -6,22292				
4	474	f(x_1) = -122,541				
5	1205	b = -3,88854				
6	2466	x_2 = -6,22292				
		n = 12				
		x_1 = -7,66562				
		f(x_1) = 138,7356				
		a = -7,66562				
		x_1 = -6,22292				
		n = 11				
		x_2 = -5,33129				
		f(x_2) = -101,071				
		a = -6,22292				
		x_1 = -5,33129				

Рисунок 2.4 – Первые 4 итерации метода Фибоначчи

Задание 4

После изучения алгоритмов всех методов и произведения вычислений в MS Excel необходимо написать код на языке Python, который будет находить точку минимума функции с точностью $\varepsilon = 0,05$ по всем трём методам, а также для удобства, можно реализовать построение графиков с показанной на них функцией и отмеченной точкой минимума. Для построения графиков будет использована библиотека «matplotlib», а для числовых вычислений (например, генерация равномерно распределённого интервала) будет использована библиотека «NumPy». Листинг кода представлен в приложении А.

Результаты работы программы представлены на рисунках 2.5 – 2.10.

```

===== МЕНЮ =====
1. Метод деления интервалов
2. Метод Фибоначчи
3. Метод золотого сечения
4. Показать график функции на отрезке [-10; 6]
5. Выход
-> 1

Результат вычисления по методу деления интервала:
Точка минимума: x_min = -6.000000
Значение в точке минимума: f(x_min) = -126.000000
Интервал (границы): [-6.015625; -5.984375]
Длина интервала: 0.031250
Заняло итераций: 9
=====

```

Рисунок 2.5 – Результат вычисления по методу деления интервалов с помощью программы на языке Python

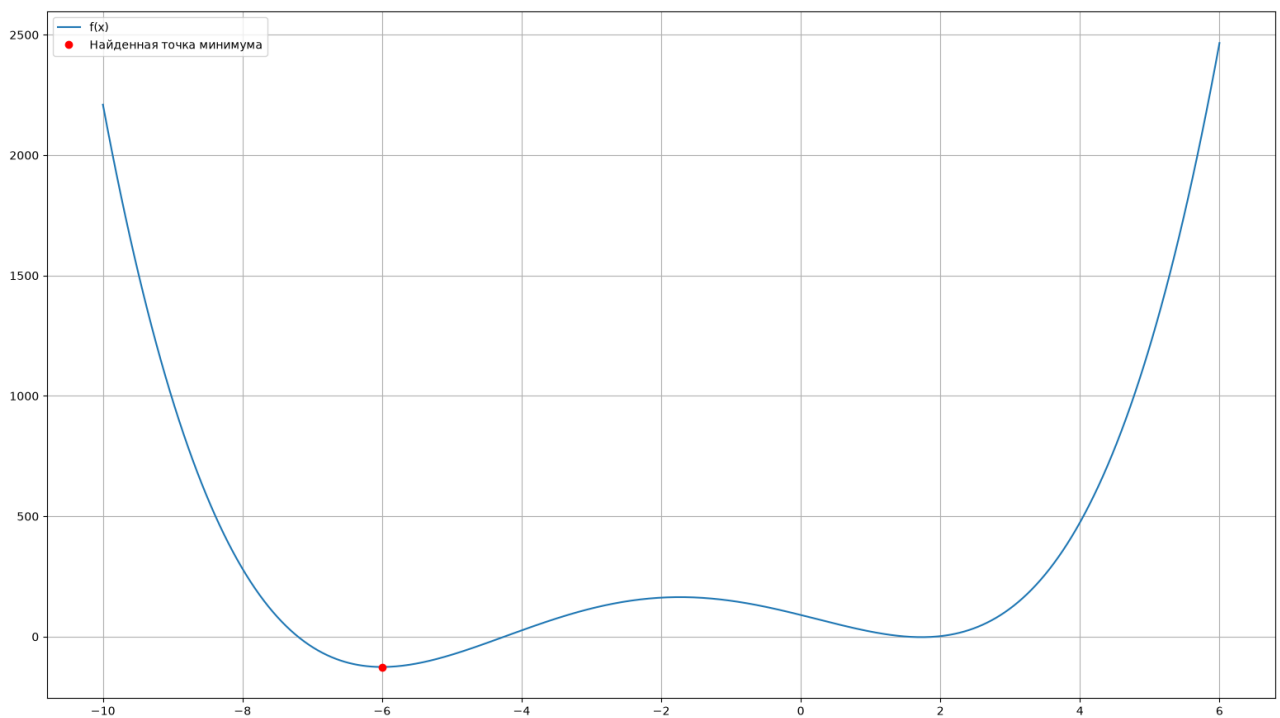


Рисунок 2.6 – График функции с найденной точкой минимума для метода деления интервалов

```

===== МЕНЮ =====
1. Метод деления интервалов
2. Метод Фибоначчи
3. Метод золотого сечения
4. Показать график функции на отрезке [-10; 6]
5. Выход
-> 2

Результат вычисления по методу Фибоначчи:
Точка минимума: x_min = -6.000000
Значение в точке минимума: f(x_min) = -126.000000
Интервал (границы): [-6.012384; -5.981424]
Длина интервала: 0.030960
Заняло итераций: 14
=====

```

Рисунок 2.7 – Результат вычисления по методу Фибоначчи с помощью программы на языке Python

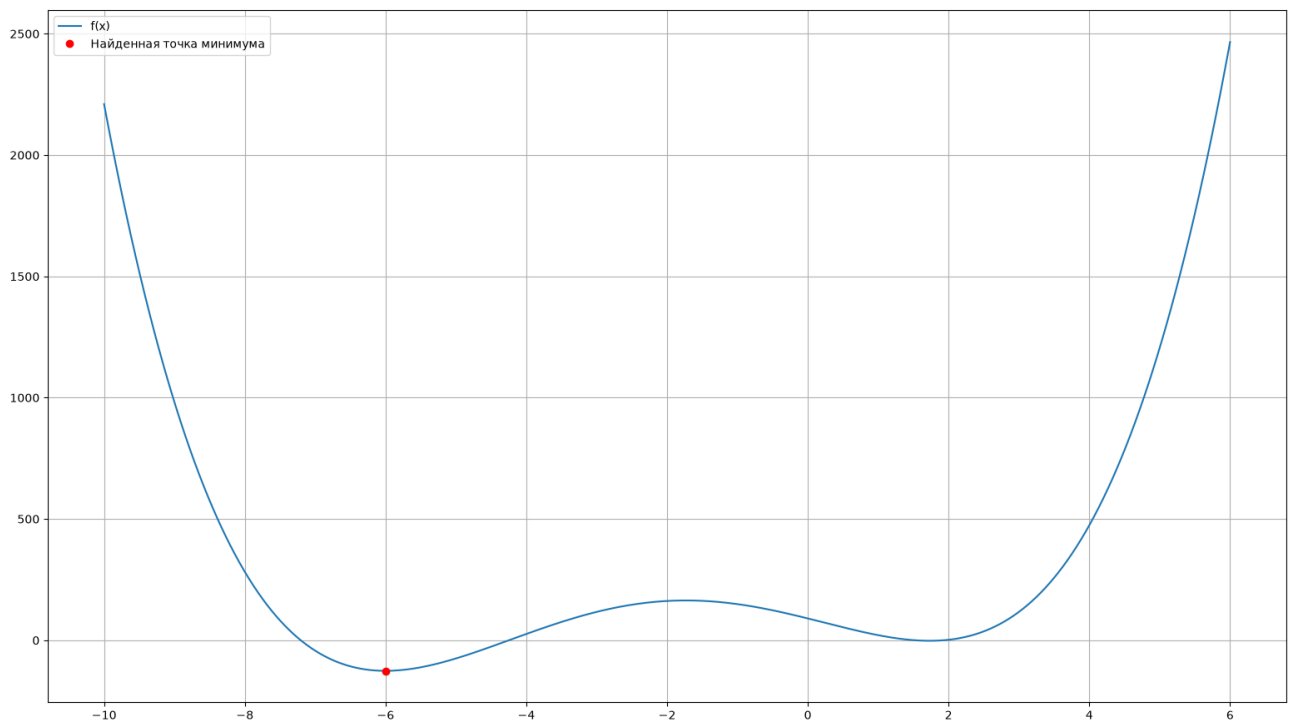


Рисунок 2.8 – График функции с найденной точкой минимума для метода Фибоначчи

```

===== МЕНЮ =====
1. Метод деления интервалов
2. Метод Фибоначчи
3. Метод золотого сечения
4. Показать график функции на отрезке [-10; 6]
5. Выход
-> 3

Результат вычисления по методу золотого сечения:
Точка минимума: x_min = -6.022640
Значение в точке минимума: f(x_min) = -125.965984
Интервал (границы): [-5.985280; -6.022640]
Длина интервала: 0.037360
Заняло итераций: 9
=====

```

Рисунок 2.9 – Результат вычисления по методу золотого сечения с помощью программы на языке Python

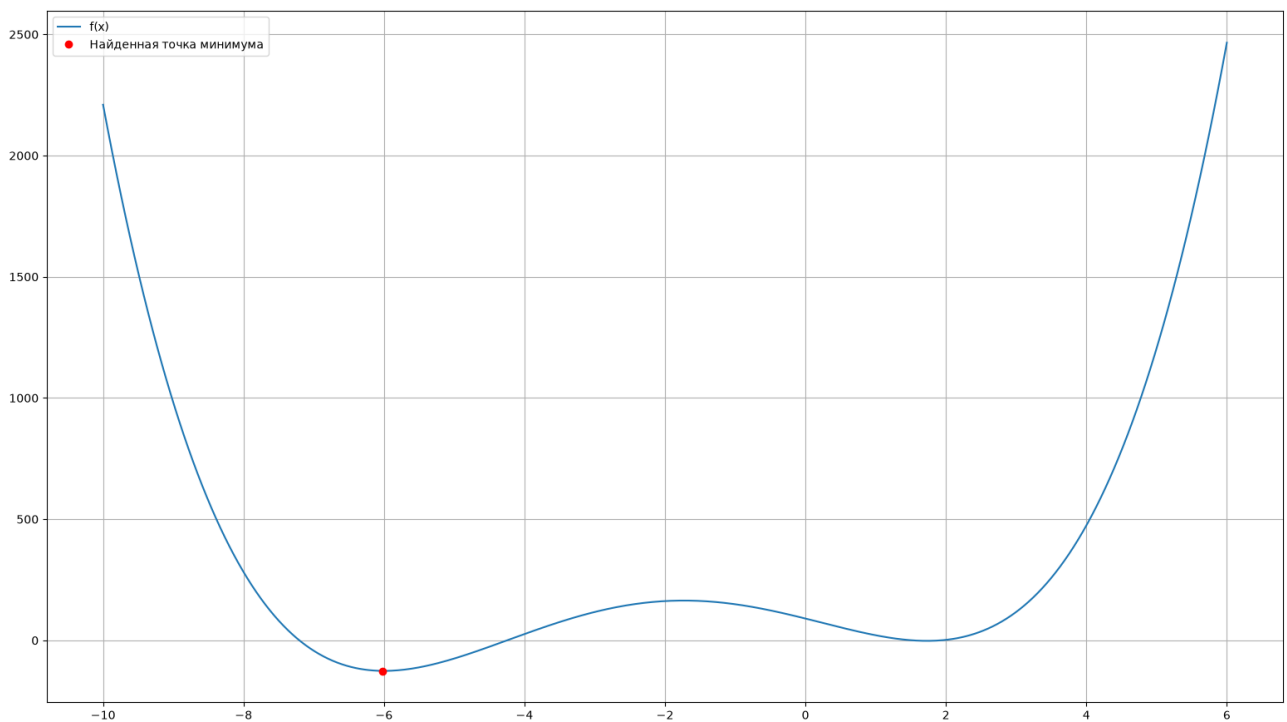


Рисунок 2.10 – График функции с найденной точкой минимума для метода золотого сечения

Задание 5

После вычисления точки минимума разными методами с помощью разных инструментов (инструментами MS Excel и с помощью языка Python) можно сделать сравнение результатов вычислений.

Сравнивая все методы можно сделать вывод, что они действительно приводят результат к теоретическому с заданной погрешностью $\varepsilon = 0,05$. Вычисления, произведённые с помощью инструментов MS Excel совпадают с решением задачи с помощью программы написанной на языке программирования Python, что видно как при аналитическом сравнении итоговых значений вычислений, так и визуальном сравнении на графиках.

ВЫВОДЫ ПО РАБОТЕ

В ходе выполнения работы были приобретены практические навыки вычисления точки минимума методами одномерной минимизации унимодальной функции (с помощью методов деления отрезка пополам, Фибоначчи и золотого сечения).

Также приобретены навыки вычисления минимума функции с помощью инструментов MS Excel и посредством написания программы на языке Python.

Все полученные значения минимума функции совпадают с теоретическим расчётом с заданной погрешностью $\varepsilon = 0,05$.

ПРИЛОЖЕНИЕ А

Текст программы на языке Python

```

import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x**4 + 8*x**3 - 6*x**2 - 72*x + 90

def viewPlot(flag = 0, x = 0, y = 0):
    arr = np.linspace(-10, 6, 500)
    plt.plot(arr, f(arr), label="f(x)")
    plt.grid(True)
    if flag == 1:
        plt.plot(x, y, 'o', color="red", label="Найденная точка минимума")
    plt.legend()
    plt.show()

def mainloop():
    eps = 0.05 # погрешность
    while True:
        print("===== МЕНЮ =====")
        print("1. Метод деления интервалов")
        print("2. Метод Фибоначчи")
        print("3. Метод золотого сечения")
        print("4. Показать график функции на отрезке [-10; 6]")
        print("5. Выход")
        c = int(input("-> "))

        if c == 5:
            return

        elif c == 4:
            viewPlot(0)

        elif c == 1:
            a = -10
            b = 6
            x_m = (a + b) / 2
            l = b - a
            iterations = 0

            while np.abs(l) >= eps:
                x_1 = a + l / 4
                x_2 = b - l / 4
                fx_1 = f(x_1)
                fx_2 = f(x_2)
                fx_m = f(x_m)
                if fx_1 < fx_m:
                    b = x_1
                elif fx_2 < fx_m:
                    a = x_2
                else:
                    a = x_1
                    b = x_2
                l = b - a
                x_m = (a + b) / 2
                iterations += 1

```

```

        l = b - a
    elif fx_2 < fx_m:
        a = x_m
        x_m = x_2
        l = b - a
    else:
        a = x_1
        b = x_2
        l = b - a
    iterations += 1

    print("\n\nРезультат вычисления по методу деления
интервала:")
    print(f"        Точка минимума функции лежит в интервале
[{a}; {b}])")
    print(f"        Длина данного интервала: L = {l}")
    print(f"        Заняло итераций: {iterations}")
    print("=====\n\n")
    viewPlot(1, x_m, fx_m)

elif c == 2:
    a = -10
    b = 6
    l = b - a
    iterations = 0
    ratio = 1 / eps
    x_min = 10**6
    f_min = x_min

    # Генерация чисел Фибоначчи
    F = [1, 1]
    while F[-1] < ratio:
        F.append(F[-1] + F[-2])
    n = len(F)
    while len(F) <= n + 2:
        F.append(F[-1] + F[-2])

    while n > 1:
        iterations += 1
        x_1 = a + (F[n] / F[n + 2]) * (b - a)
        x_2 = a + (F[n + 1] / F[n + 2]) * (b - a)
        fx_1 = f(x_1)
        fx_2 = f(x_2)
        if n == 2:
            if fx_1 <= fx_2:
                x_min = x_1
                f_min = fx_1
            else:
                x_min = x_2
                f_min = fx_2
            break
        else:

```

```

        if fx_1 <= fx_2:
            b = x_2
            x_2 = x_1
            n -= 1
            x_1 = a + (F[n] / F[n + 2]) * (b - a)
            fx_1 = f(x_1)
        else:
            a = x_1
            x_1 = x_2
            n -= 1
            x_2 = a + (F[n + 1] / F[n + 2]) * (b - a)
            fx_2 = f(x_2)

    print("\n\nРезультат вычисления по методу Фибоначчи:")
    print(f"        Точка минимума: x_min = {x_min:.6f}")
    print(f"        Значение в точке минимума: f(x_min) =
{f_min:.6f}")
    print(f"        Заняло итераций: {iterations}")
    print("=====
=====\\n\\n")
    viewPlot(1, x_min, f_min)

elif c == 3:
    tau = 0.38197 # константа
    a = -10
    b = 6
    y = a + tau * (b - a)
    z = a + b - y
    fy = f(y)
    fz = f(z)

    if fy <= fz:
        b = z
    else:
        a = y

    k = 1
    x_prev = 10**6
    iterations = 0
    max_iter = 1000 # защита от заикливания

    while iterations < max_iter:
        iterations += 1
        if fy <= fz:
            y_next = a + b - y
            fy_next = f(y_next)
            z_next = y
            fz_next = fy
        else:
            y_next = z
            fy_next = fz
            z_next = a + b - z
            fz_next = f(z_next)

```

```

    if fy_next <= fz_next:
        a_next = a
        b_next = z_next
    else:
        a_next = y_next
        b_next = b

    x_k = (a_next + b_next) / 2
    if k > 1:
        if np.abs(x_prev - x_k) <= eps:
            break

    a = a_next
    b = b_next
    y = y_next
    z = z_next
    fy = fy_next
    fz = fz_next
    x_prev = x_k
    k += 1

    x_min = x_k
    f_min = f(x_min)

    print("\n\nРезультат вычисления по методу золотого
сечения:")
    print(f"        Точка минимума: x_min = {x_min:.6f}")
    print(f"        Значение в точке минимума: f(x_min) =
{f_min:.6f}")
    print(f"        Заняло итераций: {iterations}")
    print("=====\n\n")
    viewPlot(1, x_min, f_min)

    else:
        print("\n\nНеизвестная команда\n\n")

mainloop()

```